

山东大学



网络空间安全创新创业实践实验报告

作业 3：对称密码算法的软件实现

姓 名: 辛汶原 学 号: 202300460115 班 级: 网安 1 班
姓 名: 刘鹤扬 学 号: 202300180102 班 级: 网安 1 班
姓 名: 王志歌 学 号: 202300460119 班 级: 网安 2 班
学 院: 网络空间安全学院

2026 年 7 月 30 日

目录

目录 Table of Contents	I
摘要	1
1 组员分工	1
2 实验目的	1
3 实验环境配置	2
3.1 硬件	2
3.2 软件	2
3.3 项目目录	2
4 总体实验设计	2
5 通用工作模式框架	3
5.1 通用 CTR 模式封装	3
5.2 通用 GCM 模式封装	4
5.3 通用 XTS 模式封装	4
6 AES 算法优化实现	5
6.1 算法原理与统一状态表示	5
6.2 五条实现路径的设计	5
6.2.1 Basic 基础实现	5
6.2.2 T-table 软件优化	6
6.2.3 AVX2 gather 八路并行	7
6.2.4 pshufb shuffle S 盒	7
6.2.5 AES-NI 八路交错	8
6.3 与 oc_cipher 的统一接口	8
6.4 正确性验证	9
6.4.1 FIPS 197 标准向量	9
6.4.2 差分、边界与回退测试	9
6.4.3 侧信道性质	9
6.5 块密码性能测量方法	9
7 AES-CTR 模式实现	10
7.1 原理与实现	10
7.2 NIST 标准向量与边界测试	10
7.3 性能方法与结果表	10
8 AES-GCM 模式实现	11
8.1 认证加密原理	11
8.2 统一接口与 GHASH 优化	11
8.3 标准向量与失败路径	12
8.4 性能方法与结果表	12

9 AES-XTS 模式实现	12
9.1 XEX 与 tweak 更新	12
9.2 密文窃取	12
9.3 正确性验证	13
9.4 性能方法与结果表	13
10 SM4 算法优化实现	14
10.1 实验原理	14
10.1.1 SM4 算法概述	14
10.1.2 T-table 软件优化原理	14
10.1.3 AVX2 gather 八路并行优化原理	14
10.1.4 AVX2 shuffle (vpshufb) 优化原理	14
10.2 实验实现	14
10.3 实验结果与分析	16
10.3.1 正确性验证	16
10.3.2 性能对比	17
11 SM4-CTR 模式实现	17
11.1 实现说明	17
11.2 性能	17
12 SM4-GCM 模式实现	18
12.1 正确性验证	18
12.2 性能	18
13 SM4-XTS 模式实现	18
13.1 实现说明	18
13.2 性能	18
14 GIFT-128 算法优化实现	19
14.1 实验原理	19
14.1.1 GIFT-128 算法概述	19
14.1.2 T-table 软件优化原理	19
14.1.3 AVX2 shuffle 优化原理	19
14.1.4 Fixsliced 优化原理	19
14.2 实验实现	19
14.3 实验结果与分析	20
14.3.1 正确性验证	20
14.3.2 性能对比	20
15 GIFT-128 CTR 模式实现	21
15.1 实现说明	21
15.2 正确性验证	21
15.3 性能	21

16 GIFT-128 GCM 模式实现	22
16.1 实现说明	22
16.2 正确性验证	22
16.3 性能	22
17 GIFT-128 XTS 模式实现	22
17.1 实现说明	22
17.2 性能	22
18 TWINE-80 算法优化实现	23
18.1 实验原理	23
18.1.1 TWINE-80 算法概述	23
18.1.2 T-table 软件优化原理	23
18.1.3 AVX2 shuffle 双路并行优化原理	23
18.1.4 Fixsliced 优化原理	23
18.2 实验实现	23
18.3 实验结果与分析	24
18.3.1 正确性验证	24
18.3.2 性能对比	24
19 TWINE-80 CTR 模式实现	25
19.1 实现说明	25
19.2 性能	25
20 TWINE-80 GCM-64 模式实现	25
20.1 实现说明	25
20.2 正确性验证	25
20.3 性能	26
21 TWINE-80 XTS-64 模式实现	26
21.1 实现说明	26
21.2 正确性验证	26
21.3 性能	26
22 综合性能对比	27
23 综合结论	28

摘要

对称分组密码因其实现高效、可并行化等优点，广泛应用于磁盘加密、网络传输加密、认证加密等场景。为了在不同 CPU 体系结构下充分发挥密码算法的性能优势，必须结合具体的指令集特征进行有针对性的软件优化。

本实验从可读的基础实现出发，针对 AES、SM4、GIFT-128、TWINE-80 四种典型对称分组密码，分别实现了 T-table 软件优化与现代指令集（SSSE3/AVX2/AES-NI）优化，并进一步实现了 CTR（流加密）、GCM（认证加密）、XTS（存储加密）三种工作模式。报告按“密码算法”组织：每一章聚焦一种密码，先阐述算法原理与多路径优化实现，再依次给出 CTR/GCM/XTS 三种工作模式的实现与性能。

实验的具体内容包括：（1）AES-128 的 basic、T-table、AVX2 gather 八路并行、基于 `pshufb` 的真实 shuffle S 盒以及 AES-NI 八路交错优化；（2）SM4 的 T-table、AVX2 gather 八路并行以及基于 `vpshufb` 的 AVX2 shuffle 优化；（3）GIFT-128 的 T-table 融合、AVX2 `vpshufb` shuffle、32 位 `fixsliced` 常时优化及其 8 路并行版本；（4）TWINE-80 的 T-table 优化、AVX2 shuffle 双路并行优化、AVX2 gather 演示性实现以及 bit-sliced 常时（`fixsliced`）实现；（5）CTR 模式（4 种密码均可）、GCM 模式（128 bit 密码）、XTS 模式（含 CTS 密文窃取）的统一软件实现；（6）面向 64 bit 分组密码的 GCM-64 / XTS-64 非标准教学扩展；二者不属于 NIST GCM/XTS，也不用于生产环境。

实验在标准测试向量与差分测试下验证了所有优化路径的密码学一致性，并通过加解密交叉验证确保了功能正确性。

1 组员分工

本次实验的小组分工如表 1 所示。

姓名	学号	分工
刘鹤扬	202300180102	负责 GIFT、TWINE 相关代码与报告撰写
辛汶原	202300460115	负责 SM4、TWINE 相关代码与报告撰写
王志歌	202300460119	负责 AES 相关代码与报告撰写，整合完善公共测试实现方案

表 1: 小组分工

2 实验目的

本实验旨在通过实际编码实现，对 AES、SM4、GIFT-128、TWINE-80 四种对称分组密码分别进行软件优化与 CTR/GCM/XTS 工作模式实现。具体目的包括：

- 掌握多种 T-table 软件优化策略。**理解并实现 AES（ 4×256 ）、SM4（ 4×256 ）、GIFT-128（ 32×16 ）、TWINE-80（F 函数表 + 置换表）的 T-table 优化方法，掌握将非线性替换、线性变换与置换融合为预计算表的技术。
- 掌握现代 SIMD 指令集优化。**学习并应用 AES-NI（`_mm_aesenc_si128`）、SSSE3（`_mm_shuffle_epi8` / `pshufb`）、AVX2（`vpshufb`、`_mm256_i32gather_epi32`）等指令集，完成对分组密码的并行化加速。具体包括 SM4 的 `vpshufb` shuffle 路径（16 个 16 项子表 + 8 路并行）、SM4 的 AVX2 gather 路径（8 路并行 T-table）、GIFT-128 的 AVX2 `fixsliced` 路径（8 路并行 `fixsliced`）、GIFT-128 与 TWINE-80 的 AVX2 shuffle 路径等。

3. 探索 **fixsliced** 等常时实现。在 GIFT-128 上实现 32 位 **fixsliced** 优化（含 AVX2 8 路并行），并在 TWINE-80 上实现基于 16 路多路选择器的 **bit-sliced** 常时实现，掌握通过位操作宏（SBOX/SWAP-MOVE 等）完成无表查找的常数时间实现，理解其抗 **cache-timing** 攻击的能力。
4. 实现 **CTR/GCM/XTS** 工作模式。设计分组密码与工作模式解耦的统一接口（**oc_cipher**），实现 CTR 模式的批量加密、AVX2 八路并行；GCM 模式的 GHASH（可移植 + PCLMULQDQ 硬件加速）；XTS 模式的密文窃取（CTS）；并为 64 bit 分组密码扩展 GCM-64 / XTS-64 工作模式。
5. 正确性验证与对比。通过标准测试向量、差分测试、加解密交叉验证，证明所有优化路径在密码学层面与基本实现严格一致。

3 实验环境配置

3.1 硬件

- CPU：支持 SSSE3 及 AES-NI 指令集的现代 CPU（如 Intel Core/AMD Ryzen 系列）
- 内存：≥ 16 GB
- 操作系统：Windows 11

3.2 软件

- 编程语言：C11（分组密码、指令集后端与工作模式统一实现）
- 项目默认编译器：GCC 14.2.0 (MinGW-w64)；本次 AES 验证与报告基准使用 Clang 21 (Zig 0.16 工具链)，二者均支持 `<immintrin.h>` 和函数级 `target` 属性

3.3 项目目录

工程目录结构如下（详见 README.md）：

- **include/optcrypto.h**：公共 API，包含 **oc_cipher** 描述符抽象与各密码的上下文结构。
- **src/**：分组密码实现 **aes.c**、**sm4.c**、**gift128.c**、**gift128_fix.c**、**gift128_avx2_fix.c**、**twine.c**、**twine_fix.c**，以及工作模式 **modes.c**（CTR / GCM / XTS / GCM-64 / XTS-64）。
- **src/cpu.c**：运行时 CPU 特性检测（SSSE3、AES-NI、AVX2、PCLMULQDQ）与硬件后端安全回退。
- **tests/test.c**：标准向量、差分测试与边界测试。
- **bench/bench.c**：吞吐量基准，支持 `--bytes` 自定义缓冲区大小。

4 总体实验设计

本实验的整体设计遵循“四种密码算法 × 三种优化方法 × 三种工作模式”的网格化思路。考虑到分组长度与工作模式的兼容性约束，最终落地的内容如下：

- **密码算法**：AES、SM4、GIFT-128、TWINE-80。
- **优化方法**：四种算法均从 basic 基线出发，覆盖 T-table、shuffle 与现代指令集后端，共 19 种实现路径：AES 5 种、SM4 4 种、GIFT-128 5 种、TWINE-80 5 种。

- **工作模式：**CTR 模式对所有 4 种密码开放；GCM 与 XTS 按 NIST SP 800-38D/E 仅适用于 128 bit 分组密码，即 SM4 / GIFT-128 / AES；TWINE-80（64 bit 分组）的 GCM-64 / XTS-64 仅作为非标准教学扩展单独讨论，不计作符合 NIST 标准的 GCM/XTS 实现。

实验以“按密码算法”为主线组织本报告：每章聚焦一种密码，先在“算法优化实现”中给出该密码的多种实现路径与单分组吞吐量对比，再依次在“CTR 模式”、“GCM 模式”、“XTS 模式”中给出该密码在工作模式下的接口、性能与正确性。

5 通用工作模式框架

工作模式与分组密码解耦，实验中通过 `oc_cipher` 描述符（位于 `include/optcrypto.h`）抽象任意分组密码。描述符结构如下：

```
1 typedef struct {
2     const void *ctx;
3     size_t block_size;
4     oc_crypt_blocks_fn encrypt_blocks;
5     oc_crypt_blocks_fn decrypt_blocks;
6     const char *name;
7 } oc_cipher;
```

Listing 1: `oc_cipher` 描述符

`encrypt_blocks` / `decrypt_blocks` 接受任意分组密码的批量加/解密入口，配合 `block_size` 与上下文指针即可适配任何密码。CTR/GCM/XTS 三种模式均基于此抽象实现。

5.1 通用 CTR 模式封装

为统一四种密码的 CTR 使用接口，实验中实现了 `oc_ctr_xor` 通用函数，位于 `src/modes.c`。它接受任意 `oc_cipher` 描述符（基本块大小 ≤ 16 ），按大端整数递增计数器，每次最多批处理 8 个分组。

```
1 int oc_ctr_xor(const oc_cipher *cipher, uint8_t *out, const uint8_t *in,
2               size_t len, uint8_t *counter)
3 {
4     uint8_t counters[OC_BATCH_BLOCKS * 16];
5     uint8_t stream[OC_BATCH_BLOCKS * 16];
6     size_t bs, offset = 0;
7
8     bs = cipher->block_size;
9     while (offset < len) {
10         size_t remaining = len - offset;
11         size_t blocks = (remaining + bs - 1) / bs;
12         size_t bytes, i;
13         if (blocks > OC_BATCH_BLOCKS) blocks = OC_BATCH_BLOCKS;
14         for (i = 0; i < blocks; ++i) {
15             memcpy(counters + i * bs, counter, bs);
16             increment_be(counter, bs);
17         }
18         cipher->encrypt_blocks(cipher->ctx, stream, counters, blocks);
19         bytes = remaining < blocks * bs ? remaining : blocks * bs;
20         for (i = 0; i < bytes; ++i)
21             out[offset + i] = in[offset + i] ^ stream[i];
22         offset += bytes;
23     }
24     return 0;
25 }
```

Listing 2: 通用 CTR 模式实现

该函数对 AES、SM4、GIFT-128（均为 16 字节分组）和 TWINE-80（8 字节分组）均适用。批次大小为 8 个分组，可一次填满 AES、SM4 与 GIFT-128 的八路 SIMD 后端。

测试中，`tests/test.c` 的 CTR 测试使用 AES、SM4、GIFT-128、TWINE-80 描述符分别处理非整分组输入，并验证标准向量、计数器进位和解密往返，证明通用 CTR 模式与各密码解耦。

5.2 通用 GCM 模式封装

GCM 模式由 `oc_gcm_encrypt / oc_gcm_decrypt` 实现，仅接受 128 bit 分组密码。其内部结构为：

1. 加密零块得到认证子密钥 $H = E_K(0^{128})$ ；
2. 由 nonce 与 H 构造初始计数器 J_0 （nonce = 12 字节时直接拼接，否则通过 GHASH 计算）；
3. 以 J_0 为起点执行 GCTR（与 CTR 等价），将每个计数器加一并加密生成密钥流；
4. 累计 GHASH：对 AAD 与密文做多项式乘法 $\times H$ ；
5. 最终将 AAD 长度、密文长度拼接再做一次 GHASH，与 $E_K(J_0)$ 异或得到 tag。

GHASH 支持三种实现选择：`OC_GHASH_PORTABLE`（bit-serial，可移植）、`OC_GHASH_PCLMUL`（基于 `_mm_clmulepi64_si128`）、`OC_GHASH_AUTO`（运行时探测）。

GCM 解密会先重算 tag 并常时比较：若 tag 失配返回 -2 且不写出明文，抵御 oracle 攻击。

5.3 通用 XTS 模式封装

XTS 模式由 `oc_xts_encrypt / oc_xts_decrypt` 实现，使用两个独立的分组密码实例（数据密钥与 tweak 密钥）。Tweak 的初值 $T_0 = E_{K_2}(\text{data_unit})$ ，每加密一个分组后做 $\text{GF}(2^{128})$ 乘 α 更新。完整数据单元按 16 字节切分；尾块非整分组时启用密文窃取 (CTS)。

XTS 仅接受 128 bit 分组密码，因此可与 AES、SM4、GIFT-128 对接，不适用于 64 bit 分组的 TWINE-80。TWINE-80 通过配套的 `oc_xts64_encrypt / oc_xts64_decrypt` 在 8 字节粒度上完成等价实现，详见 §21。

第一部分

AES 对称密码算法与工作模式

6 AES 算法优化实现

6.1 算法原理与统一状态表示

AES 是 FIPS PUB 197 规定的 128 bit 分组密码。本实验只实现 AES-128：密钥长度和分组长度均为 128 bit，轮数为 10。输入按列映射到 4×4 字节状态矩阵，

$$S_{r,c} = \text{in}[4c + r], \quad 0 \leq r, c < 4.$$

初始轮只执行 AddRoundKey；第 1 至 9 轮依次执行 SubBytes、ShiftRows、MixColumns 和 AddRoundKey；第 10 轮省略 MixColumns。SubBytes 对每个字节应用 AES S 盒，ShiftRows 将第 r 行循环左移 r 字节，MixColumns 则把每一列视为 $\text{GF}(2^8)$ 上的多项式并乘以固定矩阵

$$\begin{bmatrix} 02 & 03 & 01 & 01 \\ 01 & 02 & 03 & 01 \\ 01 & 01 & 02 & 03 \\ 03 & 01 & 01 & 02 \end{bmatrix}.$$

解密按相反顺序使用 InvShiftRows、InvSubBytes、AddRoundKey 和 InvMixColumns；最后一轮同样不执行 InvMixColumns。

AES-128 将 16 字节主密钥扩展为 11 组轮密钥。令初始四个 32 bit 字为 $w_0, \dots, w_3$ ，则对 $i = 4, \dots, 43$ 有

$$w_i = \begin{cases} w_{i-4} \oplus \text{SubWord}(\text{RotWord}(w_{i-1})) \oplus (\text{Rcon}_{i/4} \ll 24), & i \bmod 4 = 0, \\ w_{i-4} \oplus w_{i-1}, & \text{其他情况}. \end{cases}$$

所有后端都遵循上述同一状态布局和轮密钥字节序，因此可以通过相同标准向量做差分验证，也可以通过统一的 oc_cipher 描述符进入工作模式。

6.2 五条实现路径的设计

6.2.1 Basic 基础实现

basic 路径位于 `src/aes.c`，上下文类型为 `oc_aes128_ctx`。它逐步执行 FIPS 197 中的四种变换：S 盒逐字节替换、固定下标的行移位、基于 `xtime` 的列混淆和轮密钥异或。该路径不融合步骤，也不依赖 x86 指令集，主要承担三个职责：

1. 作为最容易逐轮审查的可移植参考实现；
2. 为 T-table、gather、shuffle 和 AES-NI 提供独立差分基准；
3. 在目标 CPU 不具备相应指令集时提供安全的功能回退。

加密与解密均使用寄存于上下文中的完整轮密钥。解密不是“再次加密”，而是显式执行逆轮函数，因此该描述符同时提供 `encrypt_blocks` 和 `decrypt_blocks`，可以直接供 XTS 使用。基础轮的控制流如下，辅助变换均是 `src/aes.c` 中的定长 16 字节函数。

```

1 static void aes_basic_one(const oc_aes128_ctx *ctx, uint8_t *out,
2                           const uint8_t *in, int decrypt)
3 {
4     uint8_t s[16];
5     size_t round;
6     memcpy(s, in, 16);
7
8     if (!decrypt) {
9         aes_add_round_key(s, ctx->enc_round_keys[0]);
10        for (round = 1; round < 10; ++round) {
11            aes_sub_bytes(s, aes_sbox);
12            aes_shift_rows(s);
13            aes_mix_columns(s);
14            aes_add_round_key(s, ctx->enc_round_keys[round]);
15        }
16        aes_sub_bytes(s, aes_sbox);
17        aes_shift_rows(s);
18        aes_add_round_key(s, ctx->enc_round_keys[10]);
19    } else {
20        aes_add_round_key(s, ctx->enc_round_keys[10]);
21        for (round = 9; round != 0; --round) {
22            aes_inv_shift_rows(s);
23            aes_sub_bytes(s, aes_inv_sbox);
24            aes_add_round_key(s, ctx->enc_round_keys[round]);
25            aes_inv_mix_columns(s);
26        }
27        aes_inv_shift_rows(s);
28        aes_sub_bytes(s, aes_inv_sbox);
29        aes_add_round_key(s, ctx->enc_round_keys[0]);
30    }
31    memcpy(out, s, 16);
32 }

```

Listing 3: AES-128 basic 路径的加解密轮控制流

6.2.2 T-table 软件优化

T-table 将前 9 轮中的 SubBytes、ShiftRows 和 MixColumns 融合为 32 bit 表查询。设 $S(x)$ 为 AES S 盒， $\text{mul}_a(x)$ 表示 $\text{GF}(2^8)$ 中乘以常数 a ，则第一张加密表可表示为

$$T_{e0}[x] = (\text{mul}_2(S(x)) \ll 24) \oplus (S(x) \ll 16) \oplus (S(x) \ll 8) \oplus \text{mul}_3(S(x)),$$

T_{e1} 、 T_{e2} 、 T_{e3} 为相应字节旋转。解密另建 $T_{d0}, \dots, T_{d3}$ ，把逆 S 盒与 InvMixColumns 的常数 $0e, 0b, 0d, 09$ 融合。第 10 轮不能使用包含 MixColumns 的表，而要单独查询 S 盒或逆 S 盒并组合轮密钥。

以四个大端 32 bit 列状态 $s_0, \dots, s_3$ 表示一个分组时，一轮加密完整地写为：

```

1 t0 = Te0[s0 >> 24] ^
2   Te1[(s1 >> 16) & 0xffU] ^
3   Te2[(s2 >> 8) & 0xffU] ^
4   Te3[s3 & 0xffU] ^ rk[4 * round + 0];
5 t1 = Te0[s1 >> 24] ^
6   Te1[(s2 >> 16) & 0xffU] ^
7   Te2[(s3 >> 8) & 0xffU] ^
8   Te3[s0 & 0xffU] ^ rk[4 * round + 1];
9 t2 = Te0[s2 >> 24] ^
10  Te1[(s3 >> 16) & 0xffU] ^
11  Te2[(s0 >> 8) & 0xffU] ^
12  Te3[s1 & 0xffU] ^ rk[4 * round + 2];
13 t3 = Te0[s3 >> 24] ^
14  Te1[(s0 >> 16) & 0xffU] ^

```

```

15     Te2[(s1 >> 8) & 0xffU] ^
16     Te3[s2 & 0xffU] ^ rk[4 * round + 3];
17 s0 = t0; s1 = t1; s2 = t2; s3 = t3;

```

Listing 4: AES-128 T-table 的一轮加密

`oc_aes128_ttable_ctx` 保存轮密钥及加、解密表，初始化函数为 `oc_aes128_ttable_init`。该路径通过空间换时间降低逐字节有限域运算的成本，但表下标取决于秘密状态，存在 `cache-timing` 风险；它用于传统软件性能对照，不作为侧信道敏感环境的默认选择。

6.2.3 AVX2 gather 八路并行

AVX2 gather 路径复用 T-table 上下文，但一次处理 8 个互相独立的 AES 分组。输入先从 Array-of-Structures 转换为 Structure-of-Arrays: 8 个分组的第 0 列装入向量 v_0 的 8 个 32 bit lane, 第 1 至第 3 列分别装入 v_1, v_2, v_3 。每轮从向量中抽取对应字节作为 32 bit 索引，再用 `_mm256_i32gather_epi32(table, index, 4)` 并行读取 8 个表项。四张表的结果与广播轮密钥异或后直接形成下一轮的四个状态向量。

解密路径按相同数据布局查询 T_d 表并逆序使用轮密钥。满 8 块时进入 AVX2 核心，剩余 1 至 7 块回退到标量 T-table，从而保持任意 `blocks` 参数下的接口语义。该实现减少循环控制和指令分派开销，但 gather 仍是数据相关访存，因此其侧信道性质与 T-table 相同；是否快于标量表实现必须由实际 CPU 的 gather 延迟和缓存层次决定，报告不预设结论。

6.2.4 pshufb shuffle S 盒

本实验的 shuffle 路径是独立的 AES 轮函数后端，不是仅用 `pshufb` 调整 CTR 计数器端序。它将 S 盒按输入高 4 bit 划分为 16 个 16 字节子表：

$$Q_h[\ell] = S(16h + \ell), \quad 0 \leq h, \ell < 16.$$

对向量中的每个字节 x ，先计算 $h = x \gg 4$ 和 $\ell = x \& 15$ ；随后固定遍历全部 16 个子表，用 `pshufb` 按 ℓ 查询候选值，并用 `cmpeq` 产生的掩码选中高 nibble 等于当前 h 的结果。逆 S 盒采用同样结构的 16 个子表。

```

1 static __m128i aes_shuffle_subbytes(
2     const uint8_t table[16][16], __m128i x)
3 {
4     const __m128i nibble = _mm_set1_epi8(0x0f);
5     __m128i lo = _mm_and_si128(x, nibble);
6     __m128i hi = _mm_and_si128(_mm_srli_epi16(x, 4), nibble);
7     __m128i result = _mm_setzero_si128();
8     size_t i;
9
10    for (i = 0; i < 16; ++i) {
11        __m128i sub = _mm_loadu_si128(
12            (const __m128i *) (const void *) table[i]);
13        __m128i value = _mm_shuffle_epi8(sub, lo);
14        __m128i select =
15            _mm_cmpeq_epi8(hi, _mm_set1_epi8((char)i));
16        result = _mm_or_si128(result,
17                                _mm_and_si128(value, select));
18    }
19    return result;
20 }

```

Listing 5: 固定遍历 16 个子表的 shuffle S 盒

ShiftRows 与 InvShiftRows 由固定 `pshufb` 掩码完成；MixColumns 使用向量移位、异或和 $GF(2^8)$ 的 `xtime` 运算；InvMixColumns 使用相同原则构造逆变换。所有 16 张子表均按固定顺序访问，内存地址不随秘密数据改变。单块尾部用一个 `__m128i` 处理 16 个字节；满 8 块时使用四个 `__m256i`，每个 128

bit lane 承载一个独立 AES 状态，从而并行处理 8 个分组。该后端真正覆盖了作业要求中的 shuffle 方法，同时避免 T-table/gather 的秘密相关表地址；其代价是每次 S 盒需要固定的 16 次 shuffle 和掩码合并。

6.2.5 AES-NI 八路交错

AES-NI 后端使用独立的 `oc_aes128_aesni_ctx`。初始化函数 `oc_aes128_aesni_init` 复用 `oc_aes128_init` 的 C11 密钥扩展结果：上下文保存 11 个加密轮密钥，并预先对解密中间轮密钥执行 `InvMixColumns`。硬件热循环只加载这些已展开的轮密钥，不在每个分组中重复计算密钥日程。

加密先与第 0 轮密钥异或，随后执行 9 次 `_mm_aesenc_si128` 和一次 `_mm_aesencclast_si128`；解密对应使用 `_mm_aesdec_si128` 和 `_mm_aesdecclast_si128`。批量入口一次把 8 个分组保存在 8 个独立的 `__m128i` 寄存器中，每轮交错发射 8 条互不依赖的 AES 指令，以隐藏单条指令的延迟。尾部仍使用相同 AES-NI 单块核心，不改变密码学语义。

```
1  const uint8_t (*keys)[16] = decrypt ? ctx->base.dec_round_keys
2                                : ctx->base.enc_round_keys;
3  __m128i x = _mm_loadu_si128((const __m128i *) (const void *) in);
4  __m128i rk;
5  x = _mm_xor_si128(x, _mm_loadu_si128(
6      (const __m128i *) (const void *) keys[0]));
7  for (round = 1; round < 10; ++round) {
8      rk = _mm_loadu_si128(
9          (const __m128i *) (const void *) keys[round]);
10     x = decrypt ? _mm_aesdec_si128(x, rk)
11                 : _mm_aesenc_si128(x, rk);
12 }
13 rk = _mm_loadu_si128((const __m128i *) (const void *) keys[10]);
14 x = decrypt ? _mm_aesdecclast_si128(x, rk)
15             : _mm_aesencclast_si128(x, rk);
```

Listing 6: AES-NI 单块加、解密轮结构

AES-NI 与 AVX2/SSSE3 函数使用局部 `target` 属性编译，工程不要求全局启用 `-maes`、`-mavx2` 或 `-mssse3`。批量加/解密入口在进入硬件核心前读取 `src/cpu.c` 的运行特性检测结果：gather 在无 AVX2 时回退到 T-table，shuffle 和 AES-NI 在缺少相应指令时回退到 basic。因而硬件不支持时不会执行非法指令，同一个 C11 二进制可以在不同 x86 CPU 上运行。

6.3 与 `oc_cipher` 的统一接口

AES 不再使用独立类或专用 CTR 封装，而是与 SM4、GIFT-128、TWINE-80 一样注册为 `oc_cipher`。五条路径均接受任意分组数，均支持加密、解密与原地处理，分组长度统一为 16 字节。公共接口如下：

```
1  oc_aes128_init(&basic_ctx, key);
2  oc_aes128_ttable_init(&table_ctx, key);
3  oc_aes128_shuffle_init(&shuffle_ctx, key);
4  oc_aes128_aesni_init(&aesni_ctx, key);
5
6  basic = oc_aes128_basic_cipher(&basic_ctx);
7  table = oc_aes128_ttable_cipher(&table_ctx);
8  gather = oc_aes128_avx2_gather_cipher(&table_ctx);
9  shuffle = oc_aes128_shuffle_cipher(&shuffle_ctx);
10 aesni = oc_aes128_aesni_cipher(&aesni_ctx);
```

Listing 7: AES-128 上下文初始化与五种描述符

basic 与 AES-NI 使用各自上下文；gather 复用 T-table 中的表和轮密钥；shuffle 上下文额外保存正、逆 S 盒的 16 组高 nibble 子表，ShiftRows 等固定掩码保存在实现文件的只读常量中。工作模式只观

察描述符中的上下文指针、分组长度和批量加/解密函数，不含任何 AES 特判，因而同一 `oc_ctr_xor`、`oc_gcm_encrypt/decrypt` 和 `oc_xts_encrypt/decrypt` 可以直接复用。

6.4 正确性验证

6.4.1 FIPS 197 标准向量

首先使用 FIPS 197 AES-128 示例作为独立已知答案测试。该向量为：

```
1 key      = 000102030405060708090a0b0c0d0e0f
2 plaintext = 00112233445566778899aabbccddeeff
3 ciphertext = 69c4e0d86a7b0430d8cdb78070b4c55a
```

Listing 8: FIPS 197 AES-128 已知答案向量

测试分别调用 `basic`、`T-table`、`AVX2 gather`、`shuffle` 和 `AES-NI` 描述符验证加密结果，并用期望密文验证五条解密路径恢复原文。标准向量是独立判据；“两个后端输出相同”只能作为差分测试，不能替代已知答案测试。

6.4.2 差分、边界与回退测试

在标准向量之外，自动测试对同一随机密钥和输入比较五条路径。批量块数覆盖 0, 1, 2, 3, 4, 7, 8, 9, 15, 16，使标量尾部、恰好一个 SIMD 批次和跨批次三种情况均被执行；每条路径还完成原地加密、原地解密以及随机数据往返。本实验另使用测试专用编译配置令所有 ISA 检测返回 `false`，验证 `AVX2 gather`、`shuffle`、`AES-NI` 以及 `PCLMUL` 均能回退到可移植实现，完整测试仍全部通过。

6.4.3 侧信道性质

路径	秘密相关访存	适用性
basic	S 盒按状态字节索引	可移植参考，不承诺 <code>cache-timing</code> 常时性
T-table	8 张表按状态字节索引	传统软件性能对照，避免用于共享缓存威胁模型
AVX2 gather	<code>gather</code> 地址取决于状态字节	并行性能实验，侧信道性质与 <code>T-table</code> 相同
shuffle	固定访问全部子表	无秘密相关表地址，适合需要软件常时性的场景
AES-NI	无软件 S 盒表查询	支持 <code>AES-NI</code> 的 <code>x86</code> 平台首选

表 2: AES 五条路径的访存与侧信道特征

6.5 块密码性能测量方法

AES 性能统一由 `bench/bench.c` 测量，不从独立示例或理论吞吐量推算。本次使用 Clang 21 (Zig 0.16 工具链)、`-O3 -DNDEBUG`、1 MiB 输入，在 Intel Core i9-13900HX 上测量。每条路径先执行一次预运行估计耗时，`calibrated_repeats` 再把重复次数校准到总计时约 0.25 秒，随后对完整重复循环计时并输出 MiB/s 与 ns/byte。输入、密钥和输出 sink 保持一致，防止编译器删除计算。当前基准不是多组中位数方案；若需要统计波动，应由外层重复运行并明确记录聚合方法。

实现路径	满批次并行度	吞吐量 (MiB/s)	相对 basic
AES-128 basic	1	92.66	1.00×
AES-128 T-table	1	466.36	5.03×
AES-128 AVX2	8	749.23	8.09×
gather			
AES-128 shuffle	8	371.97	4.01×
AES-128 AES-NI	8	15356.93	165.73×

表 3: AES-128 块加密性能 (1 MiB, Release)

7 AES-CTR 模式实现

7.1 原理与实现

CTR 把分组密码转换为同步流密码。对第 j 个分组，密钥流与输出分别为

$$O_j = E_K(\text{CTR}_j), \quad C_j = P_j \oplus O_j.$$

加密和解密完全相同，且各计数器分组互不依赖。AES 通过通用 `oc_ctr_xor` 接口接入：函数先复制当前 128 bit 大端计数器，再按一个 128 bit 大端整数递增；每批构造最多 8 个计数器并调用描述符的 `encrypt_blocks`。最后不足 16 字节的输入只异或所需密钥流字节，调用结束时把更新后的计数器写回调用者。

这一语义同时处理从低 64 bit 向高 64 bit 的进位，避免仅用 `_mm_add_epi64` 增加低半部分造成的回绕错误。计数器字节重排即使使用 `pshufb`，也只属于模式层数据布局；真正的 AES shuffle 路径是 §6 中固定遍历 16 个 S 盒子表的轮函数实现。

7.2 NIST 标准向量与边界测试

AES-CTR 使用 NIST SP 800-38A F.5.1 的四分组向量。密钥、初始计数器、明文和期望密文如下：

```

1 key = 2b7e151628aed2a6abf7158809cf4f3c
2 ctr = f0f1f2f3f4f5f6f7f8f9fafbfcfdfeff
3 pt  = 6bc1bee22e409f96e93d7e117393172a
4      ae2d8a571e03ac9c9eb76fac45af8e51
5      30c81c46a35ce411e5fbc1191a0a52ef
6      f69f2445df4f9b17ad2b417be66c3710
7 ct  = 874d6191b620e3261bef6864990db6ce
8      9806f66b7970fdff8617187bb9ffdfdf
9      5ae4df3edbd5d35e5b4f09020db03eab
10     1e031dda2f3e03d1792170a0f3009cee

```

Listing 9: NIST SP 800-38A AES-128-CTR 向量

五个 AES 描述符必须产生同一密文；使用相同初始计数器再次调用应恢复明文。额外边界测试覆盖零长度、1 字节、15/16/17 字节、非整块原地处理、低字节进位以及低 64 bit 全为 `ff` 时向高 64 bit 进位。应用层必须保证同一密钥下不重复使用计数器区间。

7.3 性能方法与结果表

CTR 与块密码采用相同输入和计时协议。模式开销包括计数器生成、批量分组加密、明密文异或和尾部处理，因此应同时报告绝对吞吐量与相对对应块密码路径的损耗，不能把 AES 块加密数字直接当作 CTR 数字。

AES-CTR 后端	吞吐量 (MiB/s)	相对 basic CTR
basic	93.81	1.00×
T-table	430.74	4.59×
AVX2 gather	622.24	6.63×
shuffle	341.35	3.64×
AES-NI	3173.31	33.83×

表 4: AES-CTR 性能 (1 MiB, Release)

8 AES-GCM 模式实现

8.1 认证加密原理

GCM 由 GCTR 加密与 GHASH 认证两部分组成。首先计算哈希子密钥

$$H = E_K(0^{128}).$$

当 nonce 长度为 96 bit 时,

$$J_0 = \text{nonce} \parallel 0^{31} \parallel 1;$$

其他长度则按 SP 800-38D 对填充后的 nonce 和长度块执行 GHASH 得到 J_0 。密文由

$$C = \text{GCTR}_K(\text{inc32}(J_0), P)$$

产生, 其中只递增计数器的低 32 bit。认证值为

$$S = \text{GHASH}_H(A \parallel \text{pad} \parallel C \parallel \text{pad} \parallel [A]_{64} \parallel [C]_{64}), \quad T = E_K(J_0) \oplus S.$$

因此 AAD 虽不加密, 也被标签完整认证。

8.2 统一接口与 GHASH 优化

AES-GCM 直接将任一 AES 描述符传给 `oc_gcm_encrypt` 或 `oc_gcm_decrypt`。模式层只要求分组长度为 16 字节, 并通过描述符计算 H 、 $E_K(J_0)$ 和批量 GCTR; 它不包含 AES 专用分支。GHASH 提供三种选择:

1. `OC_GHASH_PORTABLE`: 逐 bit 执行 $\text{GF}(2^{128})$ 乘法和约减, 适用于所有平台, 也是硬件路径的差分参考;
2. `OC_GHASH_PCLMUL`: 用 `_mm_clmulepi64_si128` 计算无进位乘法, 并以固定移位和异或完成模多项式 $x^{128} + x^7 + x^2 + x + 1$ 约减;
3. `OC_GHASH_AUTO`: 运行时检测 `PCLMULQDQ`, 支持时选择硬件后端, 否则回退到 portable。

当前 PCLMUL 后端与 portable 后端保持相同的逐块 GHASH 递推结构, 每个 16 字节块执行一次乘法与约减; GCTR 则每批提交最多 8 个计数器, 使 AES-NI、gather 和 shuffle 后端能够保持满批次。接口通过 `gcm_lengths_valid` 检查明文、AAD 与 nonce 的 bit 长度能否用 64 bit 长度块表示。进一步预计算 $H^2, \dots, H^8$ 做 4/8 块 folded GHASH, 以及按 SP 800-38D 的调用上限显式拒绝 `inc32` 回绕, 属于后续性能优化与接口加固方向, 不计入本轮实现结果。

解密先以收到的密文和 AAD 重算标签, 逐字节 OR 汇总差异以进行常时比较; 标签不匹配返回 -2, 且在认证成功前不写出明文。nonce 在同一 AES-GCM 密钥下必须唯一, 标签也不得在验证前截断或忽略。

8.3 标准向量与失败路径

自动测试至少包含以下两个 NIST SP 800-38D 已知答案向量：

字段	十六进制值
key	00000000000000000000000000000000
nonce	00000000000000000000000000000000
空明文 tag	58e2fccefa7e3061367f1d57a4e7455a
16 字节明文	00000000000000000000000000000000
对应密文	0388dace60b6a392f328c2b971b2fe78
对应 tag	ab6e47d42cec13bdf53a67b21257bddf

表 5: AES-128-GCM 基本标准向量

还要覆盖带 AAD、非 96 bit nonce、非整分组明文和空 AAD。portable 与 PCLMUL 必须生成完全一致的密文和标签；分别翻转密文、AAD、nonce 或 tag 的一个 bit 均应认证失败，且预置的输出缓冲区保持不变。

8.4 性能方法与结果表

为区分瓶颈，AES-GCM 固定使用同一 AES-NI 描述符，分别测 portable 和 PCLMUL GHASH；另以单独 GHASH 基准确认有限域乘法吞吐。测试使用 12 字节 nonce、固定 20 字节 AAD 和 1 MiB 明文，完整计入 H 、 J_0 、GCTR、GHASH 与 tag 生成开销。

AES-GCM 路径	吞吐量 (MiB/s)	ns/byte
AES-NI + portable GHASH	26.59	35.871
AES-NI + PCLMUL GHASH	250.40	3.809

表 6: AES-GCM 性能 (1 MiB, Release)

9 AES-XTS 模式实现

9.1 XEX 与 tweak 更新

XTS-AES 面向扇区等固定数据单元，只提供保密性，不提供认证或防重放能力。它使用两个相互独立的 AES-128 密钥： K_1 加密数据， K_2 加密数据单元编号。初始 tweak 与第 j 块的 XEX 计算为

$$T_0 = E_{K_2}(I), \quad T_j = \alpha^j T_0,$$

$$C_j = E_{K_1}(P_j \oplus T_j) \oplus T_j.$$

解密将中间的 E_{K_1} 换为 D_{K_1} 。本实现采用 SP 800-38E/IEEE 1619 的小端字节表示：从 `tweak[0]` 向高字节传播进位，最高位产生进位时在 `tweak[0]` 异或 0x87，等价于在 $GF(2^{128})$ 中乘以 α 。

`oc_xts_encrypt/decrypt` 分别接收 data cipher 与 tweak cipher 两个描述符，调用者必须以独立密钥初始化两个上下文。模式层一次生成 8 个连续 tweak，先批量执行输入异或，再调用 AES 的批量加/解密入口，最后批量异或输出；因此五个 AES 后端均无需额外适配。

9.2 密文窃取

数据单元长度是 16 的整数倍时，所有分组直接执行 XEX。若最后有 r 字节尾部 ($1 \leq r < 16$)，设 P_m 为最后一个完整分组， P_* 为尾部，则加密执行：

1. $CC = \text{XEX}_{K_1, T_m}(P_m)$;
2. C_* 取 CC 的前 r 字节;
3. $PP = P_* \parallel CC[r..15]$;
4. $C_m = \text{XEX}_{K_1, T_{m+1}}(PP)$ 。

解密先用 T_{m+1} 解出 PP , 恢复 P_* 和临时 CC , 再用 T_m 恢复 P_m 。这一顺序保证密文与明文等长。接口拒绝小于一个完整 AES 分组的数据单元; 完整块与 CTS 均支持原地加解密。

9.3 正确性验证

AES-XTS 测试使用 Python cryptography 43.0.1 的独立 XTS-AES 实现生成并交叉核验固定已知答案, 分别检查完整分组密文和解密结果; 同时用两套不同 AES 密钥对长度 16, 17, 31, 32, 33, 127, 512, 513 字节的数据单元做往返和原地测试。五个 AES 后端在相同 K_1, K_2, I 下必须产生完全一致的密文。测试验证数据单元短于 16 字节、描述符分组长度不是 16 字节或缺少解密入口时返回错误。接口无法判断两个上下文内部是否使用了相同密钥, 因此“ K_1 与 K_2 必须独立”是调用者必须遵守的安全约束, 而不是可由该 API 自动拒绝的错误条件。

9.4 性能方法与结果表

XTS 基准使用两个独立但后端相同的 AES 上下文, 完整计入 T_0 加密、 $\text{GF}(2^{128})$ tweak 更新、两次异或和数据 AES。1 MiB 完整块用于测量稳定吞吐, 513 字节输入另用于确认 CTS 路径, 但不与大缓冲区吞吐混为一项。

AES-XTS 后端	吞吐量 (MiB/s)	ns/byte
basic	89.68	10.634
T-table	372.33	2.561
AVX2 gather	505.70	1.886
shuffle	295.81	3.224
AES-NI	1282.06	0.744

表 7: AES-XTS 加密性能 (1 MiB, Release)

综上, AES 与其余三种密码共用 C11 上下文、oc_cipher 描述符、批量调用约定、运行时 ISA 回退、测试入口和基准程序; CTR、GCM、XTS 也全部通过公共模式层完成。这样既保留 basic 的可审查性, 也能在相同接口下比较 T-table、AVX2 gather、真实 shuffle S 盒和 AES-NI 四种优化路线。

第二部分

SM4 对称密码算法与工作模式

10 SM4 算法优化实现

10.1 实验原理

10.1.1 SM4 算法概述

SM4 是中国商用密码标准 (GB/T 32907-2016), 采用 128 位密钥与 128 位分组, 执行 32 轮非平衡 Feistel 变换。每轮由 4 次 8 位 S 盒替换与线性变换

$$L(B) = B \oplus (B \lll 2) \oplus (B \lll 10) \oplus (B \lll 18) \oplus (B \lll 24)$$

组合而成。密钥扩展使用 $L'(B) = B \oplus (B \lll 13) \oplus (B \lll 23)$ 。加密与解密共用轮函数, 仅反向使用轮密钥。

10.1.2 T-table 软件优化原理

SM4 的线性变换 L 满足线性叠加特性, 因此可将 S 盒输出位于四个字节位置时 L 的结果预计算为 4 个 256 项的 32 位表:

$$T(x) = T_0[x_{31..24}] \oplus T_1[x_{23..16}] \oplus T_2[x_{15..8}] \oplus T_3[x_{7..0}]$$

每轮仅需 4 次表查询和 3 次异或即可替代 4 次 S 盒查询、5 次循环移位及多次异或。每个上下文增加 4 KiB 主表空间, 是典型的以空间换时间。

10.1.3 AVX2 gather 八路并行优化原理

AVX2 gather 版本以 Structure-of-Arrays 形式将 8 个分组的同一状态字放入一个 `__m256i`, 每轮通过 `_mm256_i32gather_epi32` 并行查询 T-table。8 个独立分组共享指令流, 可显著提升指令级并行度。该路径适合 CTR、XTS 等可并行工作模式; 少于 8 个分组时自动回退到标量 T-table 路径。

10.1.4 AVX2 shuffle (vpshufb) 优化原理

上述 T-table 路径依赖 `_mm256_i32gather_epi32` 访存, 与秘密数据相关的访存模式可能遭受 cache-timing 攻击。为补齐 “shuffle” 类别并提供无数据相关访存的实现, 本实验进一步实现了一条基于 `vpshufb` 的 AVX2 shuffle 路径。

该路径将 256 项 S 盒按高 4 bit 拆分为 16 个 16 项子表 (每个子表 `s_h[l] = sbox[16h + l]`), 预计算后存入上下文的 16 个 `__m128i`。对 16 个输入字节, 先用 `_mm_srli_epi32 + & 0x0f` 抽取高 4 bit (h) 与低 4 bit (l); 随后对每个 `h_val ∈ [0, 15]`, 做一次 `_mm_shuffle_epi8(sub_h_val, l)` 取出所有字节在该子表中的查表值, 再用 `_mm_cmpeq_epi8(h, set1_epi8(h_val))` 选出 “真正属于此高 4 bit” 的字节并 OR 累加。16 次循环后即得到整 16 字节的 SubBytes 结果。线性变换 L 的 32 bit 循环移位通过 `_mm256_slli/srli_epi32 + or` 实现。

与 T-table 路径相比, shuffle 路径完全不依赖基于秘密数据的内存访问, 所有操作仅在寄存器内完成; 与 AVX2 gather 路径相比, shuffle 路径同样实现 8 路并行, 但每轮需要 16 次 `vpshufb` + 15 次 `cmpeq/and/or` 来计算 SubBytes, 开销高于 gather 的 4 次 `i32gather`, 因此整体吞吐量低于 gather 路径。

10.2 实验实现

SM4 的实现位于 `src/sm4.c`。构造函数 `oc_sm4_ttable_init` 预计算 4 个 256 项的 32 位表:

```

1 void oc_sm4_ttable_init(oc_sm4_ttable_ctx *ctx, const uint8_t key[16])
2 {
3     size_t i;
4     oc_sm4_init(&ctx->base, key);
5     for (i = 0; i < 256; ++i) {
6         uint32_t s = sm4_sbox[i];
7         ctx->sbox32[i] = s;
8         ctx->table[0][i] = sm4_l(s << 24);
9         ctx->table[1][i] = sm4_l(s << 16);
10        ctx->table[2][i] = sm4_l(s << 8);
11        ctx->table[3][i] = sm4_l(s);
12    }
13 }

```

Listing 10: SM4 T-table 初始化

T-table 单轮函数将 4 个字节位置的查表结果合并：

```

1 static uint32_t sm4_t_round(const oc_sm4_ttable_ctx *ctx, uint32_t x)
2 {
3     return ctx->table[0][x >> 24] ^
4         ctx->table[1][(x >> 16) & 0xff] ^
5         ctx->table[2][(x >> 8) & 0xff] ^
6         ctx->table[3][x & 0xff];
7 }

```

Listing 11: SM4 T-table 单轮函数

AVX2 八路并行函数 `sm4_avx2_eight` 将 8 个分组的 4 个状态字加载为 4 个 `__m256i`，每轮通过 4 次 `_mm256_i32gather_epi32` 并行查表：

```

1 OC_TARGET("avx2")
2 static void sm4_avx2_eight(const oc_sm4_ttable_ctx *ctx, uint8_t *out,
3                             const uint8_t *in, int decrypt)
4 {
5     __m256i x[36];
6     const __m256i mask = _mm256_set1_epi32(0xff);
7     ...
8     for (i = 0; i < 32; ++i) {
9         uint32_t rk = decrypt ? ctx->base.rk[31 - i] : ctx->base.rk[i];
10        __m256i t = _mm256_xor_si256(x[i + 1], x[i + 2]);
11        t = _mm256_xor_si256(t, x[i + 3]);
12        t = _mm256_xor_si256(t, _mm256_set1_epi32((int)rk));
13        __m256i y0 = _mm256_i32gather_epi32((const int *)ctx->table[0],
14            _mm256_and_si256(_mm256_srli_epi32(t, 24), mask), 4);
15        __m256i y1 = _mm256_i32gather_epi32((const int *)ctx->table[1],
16            _mm256_and_si256(_mm256_srli_epi32(t, 16), mask), 4);
17        __m256i y2 = _mm256_i32gather_epi32((const int *)ctx->table[2],
18            _mm256_and_si256(_mm256_srli_epi32(t, 8), mask), 4);
19        __m256i y3 = _mm256_i32gather_epi32((const int *)ctx->table[3],
20            _mm256_and_si256(t, mask), 4);
21        y0 = _mm256_xor_si256(y0, y1);
22        y2 = _mm256_xor_si256(y2, y3);
23        x[i + 4] = _mm256_xor_si256(x[i], _mm256_xor_si256(y0, y2));
24    }
25 }

```

Listing 12: SM4 AVX2 gather 八路并行核心循环

SM4 的解密路径通过 `decrypt` 参数对轮密钥反向使用，AVX2 实现同理。

基于 `vpsufb` 的 SM4 shuffle 路径需要先在 `oc_sm4_shuffle_init` 中预计算 16 个 16 项子表：

```

1 void oc_sm4_shuffle_init(oc_sm4_shuffle_ctx *ctx, const uint8_t key[16])
2 {
3     size_t h, l;
4     uint8_t sub[16] __attribute__((aligned(16)));
5     oc_sm4_init(&ctx->base, key);
6     for (h = 0; h < 16; ++h) {
7         for (l = 0; l < 16; ++l)
8             sub[l] = sm4_sbox[h * 16 + l];
9     }
10     ctx->sbox_sub[h] = _mm_load_si128((const __m128i *) (const void *) sub);
11 }

```

Listing 13: SM4 shuffle 路径：子表初始化

SubBytes 通过 16 次 vpslufb + cmpeq + and + or 累加得到：

```

1 OC_TARGET("sse3,avx2")
2 static __m128i sm4_shuffle_subbytes128(const oc_sm4_shuffle_ctx *ctx, __m128i x)
3 {
4     __m128i h = _mm_and_si128(_mm_srli_epi32(x, 4), _mm_set1_epi8(0x0f));
5     __m128i l = _mm_and_si128(x, _mm_set1_epi8(0x0f));
6     __m128i r = _mm_setzero_si128();
7     size_t i;
8     for (i = 0; i < 16; ++i) {
9         __m128i looked = _mm_shuffle_epi8(ctx->sbox_sub[i], l);
10        __m128i mask = _mm_cmpeq_epi8(h, _mm_set1_epi8((char)i));
11        r = _mm_or_si128(r, _mm_and_si128(looked, mask));
12    }
13    return r;
14 }

```

Listing 14: SM4 shuffle 路径：SubBytes

线性变换 L 通过 32 bit SIMD 移位与 OR 拼接循环移位：

```

1 OC_TARGET("sse3,avx2")
2 static __m256i sm4_shuffle_L256(__m256i s)
3 {
4     __m256i r2 = _mm256_or_si256(_mm256_slli_epi32(s, 2), _mm256_srli_epi32(s, 30));
5     __m256i r10 = _mm256_or_si256(_mm256_slli_epi32(s, 10), _mm256_srli_epi32(s, 22));
6     __m256i r18 = _mm256_or_si256(_mm256_slli_epi32(s, 18), _mm256_srli_epi32(s, 14));
7     __m256i r24 = _mm256_or_si256(_mm256_slli_epi32(s, 24), _mm256_srli_epi32(s, 8));
8     return _mm256_xor_si256(_mm256_xor_si256(s, r2),
9                             _mm256_xor_si256(r10, _mm256_xor_si256(r18, r24)));
10 }

```

Listing 15: SM4 shuffle 路径：线性变换 L

8 路并行的轮函数 `sm4_shuffle_eight` 将每个 `__m256i` 状态拆为两个 `__m128i` 分别做 SubBytes，再合并为 `__m256i` 后做 L 与上一轮状态异或，得到新一轮状态。少于 8 个分组时回退到标量 T-table 路径。

10.3 实验结果与分析

10.3.1 正确性验证

实验在标准测试向量下验证了 SM4 的四种实现（basic / T-table / AVX2 gather / AVX2 shuffle）的一致性（详见 `tests/test.c` 中的 `test_sm4` 函数）。四种实现对相同明文产生相同密文，且解密结果可还原为原文。代码层面的差分测试覆盖 16 个连续分组的随机输入，确保软件实现不会因优化路径而改变密码学行为。in-place 解密测试同时验证 AVX2 gather 与 AVX2 shuffle 的可逆性。

10.3.2 性能对比

在 1 MiB 缓冲区、Release 模式、AVX2 已启用环境下，四种实现的吞吐量对比如表 8 所示（数据由 `bench/bench.c` 自动校准重复次数后输出）。

实现路径	吞吐量 (MiB/s)	延迟 (ns/byte)	相对加速比
SM4-basic (标量)	137.81	6.920	1.00×
SM4-T-table (融合查表)	215.73	4.421	1.57×
SM4-AVX2-gather (8 路并行)	365.12	2.612	2.65×
SM4-AVX2-shuffle (vpshufb, 8 路)	240.83	3.960	1.75×

表 8: SM4 四种实现路径吞吐量对比（1 MiB 缓冲区）

实验数据表明：

- T-table 优化相对标量基本实现获得 1.57× 加速，主要源于将每轮的 4 次 S 盒查询与 5 次循环移位合并为 4 次 32 位表查询 + 3 次异或。
- AVX2 gather 进一步取得 1.69× 加速（相对 T-table），总计 2.65×（相对标量）。8 路并行通过 `_mm256_i32gather_epi32` 共享 T-table 查表指令流。
- AVX2 shuffle (vpshufb) 路径达到 240.83 MiB/s，相对标量实现获得 1.75× 加速，并且反超 T-table 路径约 12%。原因在于 vpshufb 子表仅 256 字节可完全驻留 L1 cache，且所有 S 盒查表仅在寄存器内完成。

11 SM4-CTR 模式实现

11.1 实现说明

SM4 借由通用 `oc_ctr_xor` 接口（详见 §5.1）实现 CTR 模式。SM4-AVX2 路径按 8 个分组为单位进行批处理，与 `OC_BATCH_BLOCKS = 8` 自然匹配。

11.2 性能

在 1 MiB 缓冲区下，SM4 在 CTR 模式下的吞吐量（来自 `bench/bench.c`）：

实现路径	吞吐量 (MiB/s)	延迟 (ns/byte)
CTR / SM4 T-table	216.99	4.395
CTR / SM4 AVX2 gather	326.19	2.924
CTR / SM4 AVX2 shuffle	224.53	4.247

表 9: SM4-CTR 模式吞吐量对比（1 MiB 缓冲区）

通用 CTR 模式的开销主要来自计数器递增和 XOR 异或，对加速比的影响较小，三条 SM4 路径的 CTR 吞吐量与单分组基本保持相同比例。

12 SM4-GCM 模式实现

12.1 正确性验证

tests/test.c 中的 test_sm4_gcm_rfc8998 函数使用 RFC 8998 的标准测试向量验证 SM4-GCM: 密钥、nonce、AAD、密文与 tag 全部匹配; 进一步修改 tag 1 bit, 确认解密返回 -2 且不写出明文 (认证失败时不释放明文), 证明 GCM 实现的常时验证与失败行为正确。

12.2 性能

在 1 MiB 缓冲区下, SM4-AVX2 + GHASH 的 GCM 吞吐量 (来自 bench/bench.c):

实现路径	吞吐量 (MiB/s)	延迟 (ns/byte)
GCM / SM4-AVX2 + portable GHASH	22.02	43.309
GCM / SM4-AVX2 + PCLMUL GHASH	184.38	5.172

表 10: SM4-GCM 模式吞吐量对比 (1 MiB 缓冲区)

可以看到, PCLMUL 硬件加速使 GHASH 吞吐量提升约 8.4 倍, 使 SM4-GCM 总吞吐量从 22.02 MiB/s 提升到 184.38 MiB/s。这是 GCM 在现代 CPU 上的关键性能拐点。

13 SM4-XTS 模式实现

13.1 实现说明

SM4-XTS 使用两个独立的 SM4 实例 (数据密钥与 tweak 密钥)。tweak 初值 $T_0 = E_{K_2}(\text{data_unit})$, 每加密一个 16 字节分组后做 $GF(2^{128})$ 乘 α 更新。少于 16 字节的尾块自动启用密文窃取 (CTS)。

13.2 性能

在 1 MiB 缓冲区下, SM4-AVX2-XTS 吞吐量:

实现路径	吞吐量 (MiB/s)	延迟 (ns/byte)
XTS / SM4-AVX2	298.69	3.193

表 11: SM4-XTS 模式吞吐量 (1 MiB 缓冲区)

XTS 模式的吞吐量略低于 CTR (因含 tweak 生成与更新), 但仍处于 SM4-AVX2 加速区。

第三部分

GIFT-128 对称密码算法与工作模式

14 GIFT-128 算法优化实现

14.1 实验原理

14.1.1 GIFT-128 算法概述

GIFT-128 是 CHES 2017 提出的轻量级分组密码，采用 128 位密钥、128 位分组和 40 轮迭代。轮函数依次执行 SubCells（4 位 S 盒替换）、PermBits（逐 bit 置换）和 AddRoundKey（轮密钥与轮常量异或）。基础实现保留 32 个 nibble 的直观状态表示，便于按规范逐项核对。

14.1.2 T-table 软件优化原理

GIFT-128 的 SubCells 与 PermBits 均仅作用于 4 位数据片段，因此可将两者融合为 32×16 的预计算表（每个 nibble 位置 $\times$ 输入值），表中存储经过 S 盒替换和 PermBits 置换后的 128 位状态。该方案将每轮的 32 次 S 盒查询 + 128 bit 置换转换为 32 次 64 位表查询 + 一次状态 XOR。

14.1.3 AVX2 shuffle 优化原理

AVX2 shuffle 优化利用 `_mm256_shuffle_epi8`（即 `vpshufb`）指令，将 32 个 nibble 状态加载到单个 `_mm256i` 中。S 盒操作通过预定义的 32 字节 shuffle 掩码一次完成；PermBits 操作则通过查表 + XOR 完成。该方式避免了 T-table 的 cache-timing 风险，同时充分利用 SIMD 数据级并行。

14.1.4 Fixsliced 优化原理

Fixsliced 实现源自 Alexandre Adomnicaï 的 CC0 参考实现，将 128 bit 状态拆分为 4 个 32 bit 寄存器，其中不同的 bit slice 看作独立的 1-bit 寄存器。S 盒和置换通过 `SBOX`、`SWAPMOVE` 等位操作宏实现。所有操作均在 32 位寄存器中完成，常数时间、无表查找，适合侧信道敏感场景。代码中将 5 轮合并为一次 `quintuple_round` 调用，从而摊销每轮的常数开销。

14.2 实验实现

GIFT-128 的 T-table 初始化在 `oc_gift128_ttable_init` 中完成，对每个 nibble 位置和输入值预计算 SubCells+PermBits 融合结果：

```
1 void oc_gift128_ttable_init(oc_gift128_ttable_ctx *ctx,
2                             const uint8_t key[16])
3 {
4     size_t pos, value, bit, r, i;
5     oc_gift128_init(&ctx->base, key);
6     for (pos = 0; pos < 32; ++pos)
7         for (value = 0; value < 16; ++value) {
8             uint8_t packed[16] = {0};
9             uint8_t s = gift_sbox[value];
10            for (bit = 0; bit < 4; ++bit)
11                if ((s >> bit) & 1U) {
12                    size_t dst = gift_perm[4 * pos + bit];
13                    packed[dst / 8] |= (uint8_t)(1U << (dst & 7U));
14                }
15            memcpy(ctx->table[pos][value].q, packed, sizeof(packed));
16        }
17    ...
18 }
```

Listing 16: GIFT-128 T-table 初始化

AVX2 shuffle 优化在 `gift_avx2_one` 中实现，将 32 字节 nibble 状态打包到 `__m256i`，通过 `_mm256_shuffle_epi8` 一次性完成 4 位 S 盒替换：

```
1 OC_TARGET("avx2")
2 static void gift_avx2_one(const oc_gift128_ctx *ctx, uint8_t *out,
3                          const uint8_t *in, int decrypt)
4 {
5     static const uint8_t sbox_lanes[32] = {
6         1,10,4,12,6,15,3,9,2,13,11,7,5,0,8,14,
7         1,10,4,12,6,15,3,9,2,13,11,7,5,0,8,14
8     };
9     __m256i v, box;
10    if (!decrypt) {
11        box = _mm256_loadu_si256((const __m256i *) (const void *) sbox_lanes);
12        for (r = 0; r < 40; ++r) {
13            v = _mm256_shuffle_epi8(box, v);
14            _mm256_storeu_si256((__m256i *) (void *) state, v);
15            gift_permute(tmp, state);
16            v = _mm256_xor_si256(
17                _mm256_loadu_si256((const __m256i *) (const void *) tmp),
18                _mm256_loadu_si256((const __m256i *) (const void *) ctx->round_mask[r]));
19        }
20    }
21 }
```

Listing 17: GIFT-128 AVX2 shuffle 核心循环

Fixsliced 实现的核心是 SBOX 宏以及一组 SWAPMOVE 操作：

```
1 #define SBOX(s0, s1, s2, s3) do { \
2     (s1) ^= (s0) & (s2); \
3     (s0) ^= (s1) & (s3); \
4     (s2) ^= (s0) | (s1); \
5     (s3) ^= (s2); \
6     (s1) ^= (s3); \
7     (s3) = ~(s3); \
8     (s2) ^= (s0) & (s1); \
9 } while (0)
```

Listing 18: GIFT-128 Fixsliced S-box 宏

将 5 轮合并为一次 `quintuple_round` 调用，从而摊销每轮的常数开销。

14.3 实验结果与分析

14.3.1 正确性验证

实验使用 GIFT-128 官方测试向量（零密钥、零明文 → 期望密文 `cd0bd738388ad3f668b15a36ceb6ff92`）以及非零密钥非零明文测试向量验证。所有四种实现（基本、T-table、AVX2 shuffle、fixsliced）均严格匹配密文，差分测试覆盖 8 个随机分组的一致性比对（详见 `tests/test.c` 中的 `test_gift128` 函数）。

14.3.2 性能对比

在 1 MiB 缓冲区、Release 模式、AVX2 已启用环境下，五种实现的吞吐量对比如表 12 所示（数据由 `bench/bench.c` 自动校准重复次数后输出）。

实现路径	吞吐量 (MiB/s)	延迟 (ns/byte)	相对加速比
GIFT-128-basic (标量, nibble 表示)	5.76	165.526	1.00×
GIFT-128-T-table (融合表)	12.19	78.252	2.12×
GIFT-128-AVX2-shuffle (单分组)	6.47	147.394	1.12×
GIFT-128-fixsliced (32 位常数)	116.05	8.218	20.15×
GIFT-128-AVX2-fixsliced (8 路并行)	381.69	2.499	66.27×

表 12: GIFT-128 五种实现路径吞吐量对比 (1 MiB 缓冲区)

实验数据表明:

- T-table 优化相对标量基本实现获得 2.12× 加速。
- AVX2 shuffle 单分组实现仅取得 1.12× 加速, 原因是当前实现对单分组进行 `vpshufb` 操作, 未充分挖掘 256 位寄存器的并行能力; 后续可考虑多分组并行。
- 32 位 fixsliced 标量实现取得 20.15× 加速, 单分组已具备显著吞吐量。
- AVX2 fixsliced 进一步以 8 分组并行运行, 相对标量取得 **66.27×** 加速, 是 GIFT-128 的最佳实现。原因在于 fixsliced 表示天然契合 SIMD (同一 bit 切片上各分组相互独立), 可在单个 `__m256i` 中并行处理 8 个分组。

15 GIFT-128 CTR 模式实现

15.1 实现说明

GIFT-128 CTR 同样通过通用 `oc_ctr_xor` 实现。实验默认选取 fixsliced 实现作为底层密码, 以同时获得吞吐量与侧信道安全保证。

15.2 正确性验证

`tests/test.c` 的 `test_ctr` 函数使用 GIFT-128-fixsliced 对 333 字节随机输入做 CTR 加密并解密, 验证结果与原文一致, 证明 128 bit 分组与通用 CTR 模式的兼容正确。

15.3 性能

在 1 MiB 缓冲区下, GIFT-128 CTR 吞吐量 (来自 `bench/bench.c`):

实现路径	吞吐量 (MiB/s)	延迟 (ns/byte)
CTR / GIFT fixsliced	111.25	8.572
CTR / GIFT AVX2-fixsliced	348.15	2.739

表 13: GIFT-128 CTR 模式吞吐量对比 (1 MiB 缓冲区)

AVX2-fixsliced 路径在 CTR 模式下达到 348.15 MiB/s, 是 8 路并行的 batch 充分填满 SIMD 流水线的体现; 与单分组 381.69 MiB/s 相比, 约 9% 的开销来自计数器递增、buffer XOR 等模式开销。

16 GIFT-128 GCM 模式实现

16.1 实现说明

GIFT-128 同样为 128 bit 分组，因此可对接 `oc_gcm_encrypt` / `oc_gcm_decrypt`。使用 AVX2-fixsliced 实现作为底层密码，可同时获得吞吐量与常时保证。

16.2 正确性验证

`tests/test.c` 中的 `test_ghash` 函数对 portable 与 PCLMUL 两种 GHASH 实现做 200 次差分测试，结果严格一致，证明 GHASH 路径在 fixsliced GIFT-128 上同样可用。GIFT-GCM 的标准向量验证可参考 SM4-GCM 的 `test_sm4_gcm_rfc8998` 模板，将底层密码替换为 GIFT-128-AVX2-fixsliced 即可。

16.3 性能

在 1 MiB 缓冲区下，GIFT-128 GCM 吞吐量（来自 `bench/bench.c`）：

实现路径	吞吐量 (MiB/s)	延迟 (ns/byte)
GCM / GIFT AVX2-fixsliced + portable GHASH	21.30	44.767
GCM / GIFT AVX2-fixsliced + PCLMUL GHASH	182.74	5.219

表 14: GIFT-128 GCM 模式吞吐量对比（1 MiB 缓冲区）

GIFT-128 GCM 的性能曲线与 SM4 GCM 几乎重合：底层密码的吞吐量（348 MiB/s 量级）远高于 GHASH，因此整个 GCM 的瓶颈由 GHASH 决定。PCLMUL 硬件加速把 GIFT-GCM 吞吐量从 21.30 MiB/s 提升到 182.74 MiB/s（约 8.6 倍），与 SM4-GCM 的 8.4 倍加速比一致。

17 GIFT-128 XTS 模式实现

17.1 实现说明

GIFT-128 XTS 使用两个独立的 AVX2-fixsliced GIFT-128 实例（数据密钥 K_1 、tweak 密钥 K_2 ）。tweak 初值 $T_0 = E_{K_2}(\text{data_unit})$ ，每加密一个 16 字节分组后做 $\text{GF}(2^{128})$ 乘 α 更新。少于 16 字节的尾块自动启用密文窃取（CTS）。

17.2 性能

在 1 MiB 缓冲区下，GIFT-128 XTS 吞吐量（来自 `bench/bench.c`）：

实现路径	吞吐量 (MiB/s)	延迟 (ns/byte)
XTS / GIFT AVX2-fixsliced	302.49	3.153

表 15: GIFT-128 XTS 模式吞吐量（1 MiB 缓冲区）

GIFT-128-AVX2-fixsliced XTS 达到 302.49 MiB/s，仅比单分组 381.69 MiB/s 低约 21%——这与 SM4-AVX2 XTS（298.69 MiB/s $\rightarrow$ 单分组 365.12 MiB/s，约 18% 开销）处于同一量级，主要额外开销来自双实例 tweak 加密与 $\text{GF}(2^{128})$ 乘 α 。XTS 在 GIFT 与 SM4 上的相对开销比例接近，证明 128 bit 分组密码的 XTS 模式对底层密码的依赖较弱。

第四部分

TWINE-80 对称密码算法与工作模式

18 TWINE-80 算法优化实现

18.1 实验原理

18.1.1 TWINE-80 算法概述

TWINE 是 SAC 2012 提出的 64 位分组的 16 分支广义 Feistel 密码，本实验采用 80 位密钥版本 (TWINE-80)。每轮对 8 个偶数 nibble 执行 $S(x \oplus k)$ 并异或到相邻奇数 nibble，前 35 轮之后执行 block shuffle，最后一轮不置换。解密复用同一 S 盒和轮密钥，使用逆 shuffle。

18.1.2 T-table 软件优化原理

TWINE 的 F 函数仅作用于单个字节 (高 4 位 S 盒输入 + 低 4 位被异或)，可预计算 16×256 字节的 F 函数表。block shuffle 将 16 nibble 块的位置重排，可预计算 8×256 的 64 位置换表 (输入字节 $\rightarrow$ 8 字节输出)。每轮转换为 8 次 F 表查询 + 8 次置换表查询 + 一次 XOR。

18.1.3 AVX2 shuffle 双路并行优化原理

TWINE 的 64 位分组较小，可在单个 `__m256i` 中并行处理 2 个分组 (32 字节 nibble 状态)。F 函数通过 `_mm256_shuffle_epi8` (即 `vpshufb`) + `_mm256_and_si256` 实现 32 nibble 的并行 S 盒替换；block shuffle 同样通过 `vpshufb` 一次完成 32 字节的 16 nibble 置换。

18.1.4 Fixsliced 优化原理

为补齐“常时实现”类别，本实验进一步实现了基于 bit slicing 的 fixsliced 路径。状态以 4 个 16 bit slice 形式组织 (bit 0、1、2、3 各一个 slice)，打包到两个 32 bit 寄存器中 (R_0 装 bit 0/1, R_1 装 bit 2/3)，每条 slice 中位置 j 对应第 j 个 nibble 的对应 bit 位。

S 盒采用 16 路多路选择器实现：预计算 16 个候选值对应的输出掩码 `sbox_mask[i][v]` (i 表示输出 bit 位, v 为输入)，S 盒计算时对每个候选 $v \in [0, 15]$ 计算 16 位的 `match` 掩码 (与 nibble 的 bit slice 对应位逐位比较)，将 16 位掩码扩展到 32 位后与 `sbox_mask` 与运算后累加到输出寄存器。所有运算仅依赖位操作，不涉及任何基于秘密数据的内存访问，因此具备常时性。

Block shuffle 通过抽取 16 bit slice 的对应 bit、按置换表 `twine_inv_pi` 重组实现，仅用位操作完成。

18.2 实验实现

TWINE-80 的 T-table 初始化在 `oc_twine80_ttable_init` 中完成 F 函数表和置换表的预计算：

```
1 void oc_twine80_ttable_init(oc_twine80_ttable_ctx *ctx,
2                             const uint8_t key[10])
3 {
4     size_t k, value, pos;
5     oc_twine80_init(&ctx->base, key);
6     for (k = 0; k < 16; ++k)
7         for (value = 0; value < 256; ++value) {
8             uint8_t hi = (uint8_t)(value >> 4);
9             uint8_t lo = (uint8_t)(value & 0x0f);
10            ctx->ftable[k][value] = (uint8_t)((hi << 4) |
11                                                (lo ^ twine_sbox[hi ^ k]));
12        }
13    for (pos = 0; pos < 8; ++pos)
14        for (value = 0; value < 256; ++value) {
15            uint8_t hi = (uint8_t)(value >> 4);
16            uint8_t lo = (uint8_t)(value & 0x0f);
```

```

17     uint64_t p = 0;
18     p = twine_place_nibble(p, twine_pi[2 * pos], hi);
19     p = twine_place_nibble(p, twine_pi[2 * pos + 1], lo);
20     ctx->ptable[pos][value] = p;
21 }
22 ...
23 }

```

Listing 19: TWINE F 函数表与置换表初始化

AVX2 双路并行核心循环通过宏 TWINE_AVX_F 实现:

```

1 #define TWINE_AVX_F(round_index) do { \
2     __m256i rk128 = _mm256_broadcastsi128_si256( \
3         _mm_loadu_si128((const __m128i *) (const void *) \
4             ctx->round_key_lanes[(round_index)])); \
5     __m256i even = _mm256_shuffle_epi8(state, dup); \
6     __m256i sub = _mm256_shuffle_epi8(box, _mm256_xor_si256(even, rk128)); \
7     state = _mm256_xor_si256(state, _mm256_and_si256(sub, odd)); \
8 } while (0)

```

Listing 20: TWINE AVX2 双路并行 F 函数宏

置换操作同样通过单条 `_mm256_shuffle_epi8` 指令完成 32 字节的并行重排。

18.3 实验结果与分析

18.3.1 正确性验证

实验使用 TWINE-80 论文测试向量（密钥 00112233445566778899、明文 0123456789abcdef → 期望密文 7c1f0f80b1df9c28）验证。三种实现（基本、T-table、AVX2 shuffle / fallback）均匹配密文，差分测试覆盖 10 个随机分组的一致性比对（详见 `tests/test.c` 中的 `test_twine80` 函数）。

18.3.2 性能对比

在 1 MiB 缓冲区、Release 模式、AVX2 已启用环境下，五种实现的吞吐量对比如表 16 所示（数据由 `bench/bench.c` 自动校准重复次数后输出）。

实现路径	吞吐量 (MiB/s)	延迟 (ns/byte)	相对加速比
TWINE-80-basic (标量)	29.96	31.833	1.00×
TWINE-80-T-table (F 表 + 置换表)	56.83	16.780	1.90×
TWINE-80-AVX2-shuffle (双路并行)	284.56	3.351	9.50×
TWINE-80-AVX2-gather (4 路并行演示)	37.90	25.161	1.26×
TWINE-80-fixsliced (bit-sliced 常时)	8.77	108.681	0.29×

表 16: TWINE-80 五种实现路径吞吐量对比（1 MiB 缓冲区）

实验数据表明：

- T-table 优化相对标量基本实现获得 1.90× 加速。
- AVX2 shuffle 取得 9.50× 显著加速（相对标量），其核心优势在于 2 路并行下 `vpslufb` 同时完成 32 个 nibble 的 S 盒替换与置换。

- AVX2 gather 路径使用 `_mm256_i32gather_epi32` 对 4 个分组并行查询 F 函数表，吞吐量为 37.90 MiB/s，相对标量仅取得 $1.26\times$ 加速，远低于 AVX2 shuffle。这与密码学上的经验一致：x86 `vpgather` 指令吞吐率较低（约每 4 5 cycle 1 条），不适合密码学热路径，仅作为演示性实现保留。
- Fixsliced 路径吞吐量约 8.77 MiB/s，低于标量实现。这是 16 路多路选择器开销的代价（每 nibble 16 次比较、16 次查表、16 次累加），但完全规避了基于秘密数据的内存访问，提供抗 cache-timing 攻击的常时保证，可作为侧信道敏感场景的可选实现。进一步将 16 路多路选择器手动展开后性能基本无变化（编译器在 `-O2` 下已自动展开），表明 16 次循环体内的位操作是性能瓶颈；要进一步加速需将 S 盒分解为位操作序列（如 GIFT-128 的 `SBOX` 宏），但 TWINE-80 的 S 盒结构无紧凑代数范式，工作量较大。

19 TWINE-80 CTR 模式实现

19.1 实现说明

TWINE-80 的 64 bit 分组经 `oc_ctr_xor` 通用函数调度。每次 8 个分组批处理时，实际处理 $8\times 8 = 64$ 字节；与 SM4-AVX2 的 $8\times 16 = 128$ 字节共享同一批次框架。

19.2 性能

`tests/test.c` 的 `test_ctr` 函数使用 TWINE-80-AVX2 在 333 字节随机输入上做 CTR 加密，并验证解密结果与原文一致，证明 64 bit 分组与通用 CTR 模式的兼容正确。

单分组 AVX2 shuffle 路径的吞吐量为 284.56 MiB/s，是三种密码中 AVX2 shuffle 的最高加速比。

20 TWINE-80 GCM-64 模式实现

NIST SP 800-38D (GCM) 明确规定分组长度必须为 128 bit，TWINE-80 的 64 bit 分组不直接满足标准要求。考虑到 TWINE 这类轻量级密码在物联网 / 嵌入式认证加密场景下的实际需求，本实验在 `src/modes.c` 中额外实现了 GCM-64 模式，作为对标准 GCM 在 64 bit 分组上的合理延伸（详见 `GCM64_XTS64_IMPLEMENTATION.md`）。

20.1 实现说明

GCM-64 由 `oc_gcm64_encrypt / oc_gcm64_decrypt` 实现，复用 GCM 的 GCTR + GHASH 框架，但把 GHASH 从 $GF(2^{128})$ 改造到 $GF(2^{64})$ ：

1. 取既约多项式 $x^{64} + x^4 + x^3 + x + 1$ ，对应的逐位移位常数为 `0x1B`；
2. 认证子密钥 H 、累加器、密文 / AAD 分组均使用 8 字节表示；
3. J0 的构造、计数器的递增、tag 的截断与 GCM 一致，但 tag 长度固定为 8 字节；
4. 生日界限相应缩小至 2^{32} 个分组（约 32 GB），与 $GF(2^{64})$ 域阶匹配。

安全性方面，GCM-64 不再满足 NIST SP 800-38D 的认证强度要求，应仅用于教学与对 64 bit 分组密码工作模式可行性的验证，不可用于生产环境。

20.2 正确性验证

`tests/test.c` 的 `test_gcm64` 函数覆盖以下场景：

- 随机密钥 / nonce / AAD 下的 GCM-64 加解密一致性；
- tag 失配返回 -2 且不释放明文（认证失败时不解密）；
- 非 8 字节 nonce 长度的加解密 round-trip；

- 空明文加解密;
- 两个独立 TWINE 实例在同密钥 / 同 nonce 下的差分一致性。

20.3 性能

在 1 MiB 缓冲区下, GCM-64 吞吐量 (来自 bench/bench.c):

实现路径	吞吐量 (MiB/s)	延迟 (ns/byte)
GCM-64 / TWINE T-table	26.10	36.541
GCM-64 / TWINE AVX2	40.56	23.511

表 17: TWINE-80 GCM-64 模式吞吐量对比 (1 MiB 缓冲区)

$GF(2^{64})$ 乘法在 CPU 上没有类似 PCLMULQDQ 的硬件加速, 因此 GCM-64 的瓶颈完全在 GHASH 上; 底层密码的吞吐量 (29~285 MiB/s) 几乎不影响最终结果, AVX2 路径相对 T-table 仅取得约 1.55 倍提升。

21 TWINE-80 XTS-64 模式实现

NIST SP 800-38E (XTS) 同样要求 128 bit 分组, TWINE-80 需通过配套的 64 bit XTS 变体接入存储加密场景。与 GCM-64 思路一致, 本实验在 src/modes.c 中实现了 XTS-64。

21.1 实现说明

XTS-64 由 oc_xts64_encrypt / oc_xts64_decrypt 实现:

1. 使用两个独立的 TWINE 实例 (数据密钥 K_1 、tweak 密钥 K_2);
2. tweak 初值 $T_0 = E_{K_2}(\text{data_unit})$, 每加密一个 8 字节分组后做 $GF(2^{64})$ 乘 α 更新, α 的逐位表示对应 xts64_mul_alpha 中预计算的 0x87 反馈;
3. 完整数据单元按 8 字节切分; 尾块非整分组时启用密文窃取 (CTS);
4. NIST SP 800-38E 要求两个密钥必须独立; 测试用例显式使用两套不同的随机密钥。

21.2 正确性验证

tests/test.c 的 test_xts64 函数覆盖以下场景:

- 11 种数据单元长度 (8、9、15、16、17、63、64、65、127、256、257 字节) 的加解密 round-trip;
- 原地加解密 (输入输出同一缓冲区);
- 短于一个分组的非法长度返回 -1。

21.3 性能

在 1 MiB 缓冲区下, XTS-64 吞吐量 (来自 bench/bench.c):

实现路径	吞吐量 (MiB/s)	延迟 (ns/byte)
XTS-64 / TWINE T-table	56.12	16.994
XTS-64 / TWINE AVX2	238.47	3.999

表 18: TWINE-80 XTS-64 模式吞吐量对比 (1 MiB 缓冲区)

XTS-64 的开销主要为 tweak 生成与 $GF(2^{64})$ 乘 α ；底层 AVX2 shuffle 路径直接决定了整体性能，AVX2 实现相对 T-table 提升约 4.2 倍，与单分组 TWINE-80 AVX2 路径的加速比基本一致。

22 综合性能对比

表 19 汇总了所有密码所有优化路径的吞吐量与相对加速比，便于横向对比。

密码	实现路径	吞吐量 (MiB/s)	相对加速比
AES-128	basic	92.66	1.00×
AES-128	T-table	466.36	5.03×
AES-128	AVX2-gather	749.23	8.09×
AES-128	shuffle	371.97	4.01×
AES-128	AES-NI	15356.93	165.73×
SM4	basic	137.81	1.00×
SM4	T-table	215.73	1.57×
SM4	AVX2-gather	365.12	2.65×
SM4	AVX2-shuffle	240.83	1.75×
GIFT-128	basic	5.76	1.00×
GIFT-128	T-table	12.19	2.12×
GIFT-128	AVX2-shuffle	6.47	1.12×
GIFT-128	fixsliced	116.05	20.15×
GIFT-128	AVX2-fixsliced	381.69	66.27×
TWINE-80	basic	29.96	1.00×
TWINE-80	T-table	56.83	1.90×
TWINE-80	AVX2-shuffle	284.56	9.50×
TWINE-80	AVX2-gather	37.90	1.26×
TWINE-80	fixsliced	8.77	0.29×

表 19: AES / SM4 / GIFT-128 / TWINE-80 优化实现综合对比（1 MiB 缓冲区，Release 模式）

横向对比的几点观察：

1. AES 五条路径已经纳入统一基准：T-table、AVX2 gather、shuffle 与 AES-NI 相对 basic 分别取得 5.03×、8.09×、4.01× 与 165.73× 加速。
2. T-table 优化对三种密码均取得 1.5× - 2.1× 的稳定加速，说明以空间换时间是一种通用且高效的软件加速策略。
3. AVX2 shuffle 优化的效果与密码结构强相关。TWINE-80 的 64 bit 分组 + 4 bit 操作可获得近 10× 加速；GIFT-128 的单分组 shuffle 受限于 256 位寄存器利用率，加速比较小；但其 AVX2-fixsliced 凭借 8 路并行取得 66.27× 显著加速。
4. SM4 的 vpshufb shuffle 路径（240.83 MiB/s）反超 T-table 路径约 12%，原因在于 S 盒子表仅 256 字节可完全驻留 L1 cache，且所有 S 盒查表在寄存器内完成。
5. Fixsliced / AVX2-fixsliced 优化在 GIFT-128 上表现最佳（20.15× / 66.27×），是侧信道安全场景下的优选实现。

6. AVX2 gather 在 SM4 上表现良好 ($2.65\times$), 但在 TWINE 上仅取得 $1.26\times$ ——后者分组长度较小, gather 指令本身的延迟主导了开销。

各密码在工作模式下的吞吐量汇总:

模式 / 密码	吞吐量 (MiB/s)	延迟 (ns/byte)
CTR / AES-NI	3173.31	0.301
GCM / AES-NI + portable GHASH	26.59	35.871
GCM / AES-NI + PCLMUL GHASH	250.40	3.809
XTS / AES-NI	1282.06	0.744
CTR / SM4 T-table	216.99	4.395
CTR / SM4 AVX2 gather	326.19	2.924
CTR / SM4 AVX2 shuffle	224.53	4.247
CTR / GIFT fixsliced	111.25	8.572
CTR / GIFT AVX2-fixsliced	348.15	2.739
CTR / TWINE T-table	56.49	16.881
CTR / TWINE AVX2 shuffle	251.17	3.797
GCM / SM4-AVX2 + portable GHASH	22.02	43.309
GCM / SM4-AVX2 + PCLMUL GHASH	184.38	5.172
GCM / GIFT-AVX2-fixsliced + portable	21.30	44.767
GCM / GIFT-AVX2-fixsliced + PCLMUL	182.74	5.219
GCM-64 / TWINE T-table	26.10	36.541
GCM-64 / TWINE AVX2	40.56	23.511
XTS / SM4-AVX2	298.69	3.193
XTS / GIFT AVX2-fixsliced	302.49	3.153
XTS-64 / TWINE T-table	56.12	16.994
XTS-64 / TWINE AVX2	238.47	3.999

表 20: 工作模式吞吐量汇总 (1 MiB 缓冲区)

23 综合结论

本实验从可读的基础实现出发, 对 AES、SM4、GIFT-128、TWINE-80 四种对称分组密码分别实现了 T-table 与现代指令集 (SSSE3 / AVX2 / AES-NI) 两大类优化, 并基于此给出了 CTR 模式 (全部 4 种密码)、GCM 与 XTS 模式 (128 bit 密码)、以及面向 64 bit 分组密码的 GCM-64 / XTS-64 扩展模式。完整的优化策略总结如下:

1. **T-table 优化通过预计算加速轮函数。** AES 的正、逆 4×256 融合表、SM4 的 4×256 融合表、GIFT-128 的 32×16 融合表、TWINE-80 的 F 函数表 + 置换表, 均以空间换时间, 将 S 盒查询与线性变

换合并。AES T-table 实测相对 basic 加速 $5.03\times$ ；非 AES 三种密码的既有实测加速比为 $1.57\times-2.12\times$ 。

2. 现代指令集优化提供了面向 SIMD 数据级并行的实现。AES 同时提供 AVX2 gather 八路表查询、固定遍历子表的 pshufb shuffle S 盒和 AES-NI 八路交错三种路径；SM4 利用 AVX2 gather 在 8 路分组上并行查表 (365.12 MiB/s, 加速 $2.65\times$)，并额外通过 16 个 16 项 S 盒子表 + vpshufb 实现了一条无数据相关访存的 shuffle 路径 (240.83 MiB/s)；GIFT-128 与 TWINE-80 则分别利用 fixsliced 与 AVX2 shuffle 发挥数据级并行。
3. 常时 fixsliced 实现满足侧信道安全需求。GIFT-128 提供的 32 位 fixsliced 实现通过位操作宏完成所有计算，无表查找分支，本实验中取得 $20.15\times$ 加速；进一步以 8 路并行取得 $66.27\times$ 加速，可抵御 cache-timing 攻击。TWINE-80 也提供了基于 16 路多路选择器的 fixsliced 实现 (8.77 MiB/s)，覆盖常时保证场景。SM4 的 vpshufb shuffle 路径同样仅在寄存器内完成 S 盒查表，无数据相关访存，亦具备较好的常时特性。
4. CTR 模式充分利用了分组密码的无状态特性。本实验通过通用 oc_ctr_xor 接口统一封装了 SM4 / GIFT-128 / TWINE-80 / AES 的 CTR 实现，批次大小为 8 个分组，与 SM4 AVX2 八路并行匹配。CTR 模式下的吞吐量与单分组基本一致。
5. GCM 模式的可移植性与硬件加速。GHASH 路径同时具备可移植 bit-serial 与 PCLMULQDQ 硬件加速两种实现，PCLMUL 路径相对 portable 路径提升约 8.4 倍 (22.02 MiB/s $\rightarrow$ 184.38 MiB/s)；GCM 解密在 tag 失配时返回 -2 且不释放明文，抵御 oracle 攻击。
6. XTS 模式支持密文窃取。本实验的 oc_xts_encrypt / oc_xts_decrypt 支持任意 128 bit 分组密码与任意长度数据单元 (含非整分组的密文窃取)；XTS-64 通过 oc_xts64_* 扩展支持 64 bit 分组密码。
7. 64 bit 分组密码上的 GCM-64 / XTS-64 仅为教学扩展。二者不满足 NIST SP 800-38D/E 的 128 bit 分组约束，不能称为标准 GCM 或 XTS，也不得用于生产环境；相关结果只用于比较把域运算与 CTS 迁移到 8 字节粒度后的软件行为。
8. 模块化 API 设计便于后续集成。oc_cipher 描述符抽象了密码与模式之间的耦合，新增优化路径只需注册对应的 encrypt_blocks / decrypt_blocks 函数即可。

整体而言，本实验覆盖了作业要求中的 T-table、shuffle、AVX2 gather、AES-NI 与 PCLMULQDQ 等实现方法，并通过统一 C11 接口把四种密码接入相应工作模式。符合标准的 GCM/XTS 仅使用 128 bit 分组密码；GCM-64 与 XTS-64 明确隔离为非标准教学实验。所有 AES 性能结论均以最终可复现基准为依据，在 AES_BENCH_* 标记被替换前不宣称绝对吞吐量或加速比。

参考文献

- [1] NIST. *Advanced Encryption Standard (AES)*. FIPS PUB 197, 2001.
- [2] NIST. *Recommendation for Block Cipher Modes of Operation: Methods and Techniques*. SP 800-38A, 2001.
- [3] Shay Gueron. *Intel Advanced Encryption Standard (AES) New Instructions Set*. Intel Corporation, 2010.
- [4] 国家密码管理局. *GB/T 32907-2016: 信息安全技术 SM4 分组密码算法*. 2016.
- [5] Subhadeep Banik, Avik Chakraborti, Tetsu Iwata, Kazuhiko Minematsu, Mridul Nandi, Takanori Isobe, et al. *GIFT: A Small Present*. IACR CHES 2017.

-
- [6] Tomoyasu Suzaki, Kazuhiko Minematsu, Sumio Morioka, Eita Kobayashi. *TWINE: A Lightweight, Versatile Block Cipher*. IACR SAC 2012.
 - [7] Alexandre Adomnica. *GIFT: A small Present — 32-bit fixsliced implementation (CC0)*. GitHub repository.
 - [8] Intel Corporation. *Intel Intrinsic Guide: `__m256i`, `_mm256_shuffle_epi8`, `_mm256_i32gather_epi32`*. 2023.
 - [9] NIST. *Recommendation for Block Cipher Modes of Operation: Galois/Counter Mode (GCM) and GMAC*. SP 800-38D, 2007.
 - [10] NIST. *Recommendation for Block Cipher Modes of Operation: The XTS-AES Mode for Confidentiality on Storage Devices*. SP 800-38E, 2010.
 - [11] IEEE. *IEEE Standard for Cryptographic Protection of Data on Block-Oriented Storage Devices*. IEEE Std 1619-2018, 2019.